Table des matières

Labo 12 — Rebâtir le pipeline de bout en bout 1
Avant de commencer — la plateforme du capstone 2
Étape 1 — Les séries de l'usine, une étape du pipeline 4
Étape 2 — Le contrat de l'argent en 1.1.0, et le fuseau du conteneur 7
Étape 3 — Le DAG de la reconstruction 10
Étape 4 — Mettre de côté, puis effacer 12
Étape 5 — Refaire 13
Étape 6 — Trois preuves 14
Étape 7 — Le chemin chaud, de zéro 19
Étape 8 — Le tableau de bord de l'usine 20
Étape 9 — L'incident, de bout en bout 25
Étape 10 — Le `README.md` du dépôt 27
Étape 11 — Versionner, et étiqueter la version 1.0 30
Récapitulatif 30
Dépannage 32

Labo 12 — Rebâtir le pipeline de bout en bout

Labo · Cours 12/12 — Formation Data Engineering

Ce que vous allez faire. Faire de la plateforme de Boréal quelque chose qu'on peut effacer sans crainte. Ajouter au pipeline l'étape qui manquait pour tout refaire d'une commande, les séries de l'usine ; corriger le contrat qui jugeait autrement dans un conteneur ; écrire un DAG qui refait tout le chemin froid. Puis mettre l'entrepôt de côté, effacer le lac et l'entrepôt, tout refaire, et le prouver de trois façons. À la seconde séance, rejouer le chemin chaud de zéro, monter le tableau de bord de l'usine dans Grafana, remonter l'incident de l'automne 2025 depuis l'amont, et livrer : le `README.md` du dépôt, un commit, une étiquette `v1.0`. Objectif de fin : une plateforme refaite en un peu plus de trois minutes, 26 tables et 0 écart avec celle des onze premiers cours, et l'alerte de vibration revenue à la même minute.

Le cours compte deux séances. La première va jusqu'à l'étape 6 et tient le chemin froid : elle se tient seule, et son point d'arrêt est marqué. La seconde tient le chemin chaud, le tableau de bord, l'incident et la livraison.

© 2026 iAlexMG. Tous droits réservés.

Trois terminaux servent, et ce labo les emploie tous. L'Anaconda Prompt, environnement `formations` actif, lance Python, Git et les copies de fichiers : depuis `Data_Domain\Projet_Boreal_intermediaire` pour ce qui touche au projet entier, depuis `Projet_Boreal_intermediaire\01_de` pour le pipeline et le chemin chaud, depuis `Data_Domain` pour le kit et le rejoueur. La ligne de commande DuckDB, lancée depuis `Data_Domain\Projet_Boreal_intermediaire`, lit l'entrepôt. Ubuntu, dans WSL 2, sert à Docker, à Airflow et à la base de séries temporelles, depuis `Projet_Boreal_intermediaire/plateforme`. L'entrepôt n'accepte qu'un processus à la fois : quittez la ligne de commande DuckDB par `.exit` avant chaque commande du pipeline ou d'Airflow qui l'écrit.

Le journal du pipeline est montré sans la date et l'heure qui ouvrent chacune de ses lignes, et un chemin de votre poste est abrégé en `C:\…`.

La liste de contrôle :

- ☐ La plateforme recopiée, l'image d'Airflow refaite avec le pilote ODBC, Grafana démarré, la base de l'ERP rechargée à mars 2026 : `VBAK` 27 185, `ZSERIE` 29 782
- ☐ L'étape `argent` dans le pipeline : 290 567 lignes, sans commande à la main
- ☐ Le contrat de l'argent en 1.1.0 : avril 2026 rompu sur 30 jours sous Windows comme dans le conteneur, le 31 mars respecté des deux côtés
- ☐ Le DAG `boreal_reconstruction` lu par Airflow : sept tâches
- ☐ L'entrepôt mis de côté dans `plateforme/avant/`, le lac effacé, `series` arrêté
- ☐ La reconstruction au vert, en un peu plus de trois minutes
- ☐ Les trois preuves : le kit conforme ; 26 tables et 0 écart ; l'ERP à 0 écart dans les deux sens
- ☐ Le chemin chaud rejoué de zéro : 62 726 points, l'alerte levée le 2025-10-22 à 8 h 15
- ☐ Le tableau de bord de l'usine dans Grafana, ses quatre panneaux remplis
- ☐ Les trois faits de l'incident, lus dans ce que la plateforme livre
- ☐ Le `README.md` du dépôt écrit, le travail commité et étiqueté `v1.0`

Avant de commencer — la plateforme du capstone

Ce cours allume le dernier service de la plateforme, Grafana, et en change deux fichiers : l'image d'Airflow reçoit le pilote ODBC de PostgreSQL, pour que l'extraction de l'ERP tourne aussi dans un DAG, et le compose reçoit le service `grafana` et l'adresse de la base de l'ERP vue depuis les conteneurs. Grafana demande en plus deux fichiers de réglages, dans un dossier `grafana/`. Recopiez les trois depuis `_univers/plateforme/`, dans l'Anaconda Prompt, depuis `Data_Domain\Projet_Boreal_intermediaire` :

© 2026 iAlexMG. Tous droits réservés.

```
copy /Y ..\_univers\plateforme\docker-compose.yml plateforme\
copy /Y ..\_univers\plateforme\Dockerfile.airflow plateforme\
xcopy /E /I /Y ..\_univers\plateforme\grafana plateforme\grafana
```

`/Y` remplace sans demander un fichier qui existe déjà. Les deux premières commandes répondent `1 fichier(s) copié(s).`, la troisième nomme `sources.yaml` et `tableaux.yaml`, puis `2 fichier(s) copié(s)`. Ouvrez ensuite le `.env` de la plateforme, `notepad plateforme\.env`, et ajoutez sous les deux mots de passe qui s'y trouvent celui du compte d'administration de Grafana :

```
GRAFANA_MOT_DE_PASSE=choisissez-en-un
```

Lettres et chiffres seulement, comme les deux autres, et notez-le : l'étape 8 le demande. Grafana ne l'applique qu'à la création de son volume, et le changer ensuite ne change rien au compte.

Passez dans Ubuntu, dans le dossier de la plateforme :

```
cd plateforme
wsl
```

Refaites l'image d'Airflow, puis démarrez tout :

```
docker compose build airflow-init
docker compose up -d
docker compose ps
```

La construction ajoute une couche à l'image du Cours 06 : le pilote, installé en `root` par le gestionnaire de paquets de Debian, puis `pyodbc`. Comptez une quinzaine de secondes, et une image qui passe de 3,18 à 3,27 Go. `docker compose up -d` recrée les cinq services d'Airflow, dont l'environnement a changé, et démarre Grafana. Une minute plus tard :

```
SERVICE                STATUS
airflow-apiserver      Up 45 seconds (healthy)
airflow-dag-processor  Up 46 seconds (healthy)
airflow-db             Up About a minute (healthy)
airflow-scheduler      Up 46 seconds (healthy)
airflow-triggerer      Up 45 seconds (healthy)
erp                    Up About a minute
grafana                Up About a minute
kafka                  Up About a minute
mqtt                   Up About a minute
series                 Up About a minute
```

La base de l'ERP date du Cours 04 : chargée depuis la génération par défaut, elle s'arrête au 31 décembre 2025. Les sources, régénérées au Cours 06, vont jusqu'au 31 mars 2026, et la copie de la base que l'ERP livre dans `sources/erp/postgresql/` aussi. La reconstruction relira la base : rechargez-la à la date des sources, avec le script du Cours 04.

© 2026 iAlexMG. Tous droits réservés.

```
docker compose exec -w /erp erp psql -U postgres -d erp_boreal -f charger_erp.sql
```

```
 table_sap | lignes
-----------+--------
 CDHDR     |     53
 CDPOS     |     53
 KNA1      |    156
 MAKT      |    136
 MARA      |     68
 T005U     |     20
 VBAK      |  27185
 VBAP      |  82842
 ZSERIE    |  29782
(9 rows)
```

Trois mois de plus qu'au Cours 04 : 1 702 commandes, 5 110 postes et 2 007 numéros de série. Vérifiez enfin que l'image porte le pilote :

```
docker compose exec airflow-scheduler python -c "import pyodbc; print(pyodbc.drivers())"
```

```
['ODBC Driver 18 for SQL Server', 'PostgreSQL ANSI', 'PostgreSQL Unicode']
```

Le premier vient de l'image officielle d'Airflow ; les deux autres sont ceux que vous venez d'ajouter. `PostgreSQL Unicode` est le nom que le module `erp.py` du Cours 04 cherche, le même que sous Windows. Gardez ce terminal Ubuntu ouvert pendant toute la séance.

Étape 1 — Les séries de l'usine, une étape du pipeline

Théorie : `Theorie_12_DE.md`, section 4.

Au Cours 08, les séries de l'usine se refaisaient par une commande à part, `duckdb -f 01_de/sql/argent_usine.sql`, lancée depuis `Projet_Boreal_intermediaire`. Une reconstruction ne peut pas demander une commande à la main au milieu : les séries deviennent une étape du pipeline, que le DAG appellera comme les autres. Le script, lui, ne change pas, et c'est là que tombe le premier piège. Il commence par attacher l'entrepôt, parce qu'on le lance depuis une base vide ; le pipeline, lui, tient déjà l'entrepôt ouvert. Voyez ce que DuckDB en pense, dans l'Anaconda Prompt, depuis `Projet_Boreal_intermediaire` :

```
duckdb plateforme/entrepot/boreal.duckdb -c "ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot;"
```

```
Binder Error: Unique file handle conflict: Cannot attach "entrepot" - the database file
"C:\…\Projet_Boreal_intermediaire\plateforme\entrepot\boreal.duckdb" is already attached by database "boreal"
```

La ligne de commande a ouvert l'entrepôt comme base principale, sous le nom `boreal`, et refuse de l'attacher une seconde fois, même avec `IF NOT EXISTS`, qui ne regarde que le nom. C'est ce que ferait le script lancé tel quel par le pipeline. Deux sorties : écrire un second script sans `ATTACH`, qui divergerait du premier au

© 2026 iAlexMG. Tous droits réservés.

premier changement, ou faire adapter le script par l'étape, en vérifiant qu'elle trouve bien ce qu'elle retire. C'est la seconde. Passez dans le dépôt, `cd 01_de`, puis :

```
notepad pipeline\argent.py
```

Ce code ne se tape pas dans la ligne de commande DuckDB : c'est un fichier du dépôt. Au moment d'enregistrer, choisissez « Tous les fichiers » dans la liste des types.

```
# -*- coding: utf-8 -*-
"""argent.py : les séries de l'usine du Cours 08, refaites par le pipeline (Cours 12)."""
from .config import DEPOT, chemin
from .journal import LOG

ATTACHE = "ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot;"
TABLES = ("usine_horaire", "production_horaire", "energie_horaire")


class ScriptInattendu(RuntimeError):
    """Le script de l'argent n'a plus la forme que l'étape sait adapter."""


def adapter(texte: str) -> str:
    """Le script du Cours 08, prêt pour une connexion qui a déjà l'entrepôt comme base principale."""
    if texte.count(ATTACHE) != 1:
        raise ScriptInattendu("argent_usine.sql n'attache plus l'entrepôt comme au Cours 08 : "
                              "relire le script avant de changer cette étape.")
    return texte.replace(ATTACHE, "").replace("entrepot.argent", "argent")


def executer(con, fenetre=None, livraisons=None) -> int:
    """Refait les séries de l'usine. Rend le total des lignes des trois tables.

    Pas de fenêtre ici : le script refait les trois tables entières, par CREATE OR REPLACE, et
    réécrit leurs partitions du lac avec OVERWRITE. Il suppose le bronze de l'historien en place.
    """
    (chemin("lac") / "argent" / "usine").mkdir(parents=True, exist_ok=True)
    con.execute(adapter((DEPOT / "sql" / "argent_usine.sql").read_text(encoding="utf-8")))
    total = 0
    for table in TABLES:
        lignes = con.execute(f"SELECT COUNT(*) FROM argent.{table}").fetchone()[0]
        LOG.info("  argent.%-20s %10s lignes", table, f"{lignes:,}".replace(",", " "))
        total += lignes
    return total
```


© 2026 iAlexMG. Tous droits réservés.

`adapter` retire la ligne de l'`ATTACH` et le préfixe `entrepot.` des noms de tables, pour que `entrepot.argent.usine_horaire` devienne `argent.usine_horaire`, dans l'entrepôt que la connexion tient déjà. Il ne retire rien à l'aveugle : si le script ne contient plus exactement une fois la ligne attendue, l'étape s'arrête sur `ScriptInattendu`, plutôt que d'exécuter un script qu'elle ne comprend plus. Le `mkdir` du dossier du lac n'est pas décoratif : `COPY … PARTITION_BY` crée le dossier final, pas ses parents (Cours 02), et le Cours 08 vous l'avait fait créer à la main. Une reconstruction qui efface le lac doit savoir le recréer.

Branchez l'étape dans la porte d'entrée. Ouvrez `pipeline\__main__.py` et changez trois choses, comme au Cours 11. La ligne d'import prend le module :

```
from . import argent, bronze, contrats, corpus, erp, meteo, raw
```

Le dictionnaire `ETAPES` gagne une ligne, sous celle de `meteo` :

```
    "argent": (argent.executer, "les séries horaires de l'usine, refaites depuis le bronze", False,
               False),
```

Hors de « tout », et sans fenêtre : le script refait les trois tables entières. Et le paragraphe de la docstring qui disait trois étapes hors de « tout » en dit maintenant quatre, avec la raison de la quatrième :

```
Quatre étapes ne sont pas dans « tout » : erp, qui demande le conteneur de la base et le pilote
psqlODBC ; meteo, qui demande le réseau ; contrats (Cours 11), qui juge ce que les autres ont écrit
et demande PyYAML ; et argent (Cours 12), qui refait les séries de l'usine après le bronze. Un
pipeline rejouable hors ligne, avec DuckDB et la bibliothèque standard pour seules dépendances, ne
suppose pas les trois premières. La quatrième n'en demande pas plus que lui, mais « tout » reste le
chemin du Cours 05, celui que le kit de départ d'Analytics Engineering rejoue avant de refaire
lui-même les séries : l'y ajouter ferait tourner deux fois le même script.
```

Lancez l'étape :

```
python -m pipeline --etapes
python -m pipeline argent
```

```
Étapes du pipeline :
  bronze   les CSV et le JSON de l'usine, rangés en Parquet   (reconstruit tout)
  raw      les 22 livraisons chargées dans la couche RAW   (fenêtre)
  corpus   la documentation déposée telle quelle dans le bronze   (reconstruit tout)
  erp      les neuf tables de la base de l'ERP, par ODBC   (hors de « tout », reconstruit tout)
  meteo    les observations d'ECCC, par son API   (hors de « tout », reconstruit tout)
  argent   les séries horaires de l'usine, refaites depuis le bronze   (hors de « tout », reconstruit tout)
  contrats les contrats de données, jugés sur ce qui est arrivé   (hors de « tout », fenêtre)
INFO    pipeline : pipeline : argent, depuis C:\…\Data_Domain\Projet_Boreal_intermediaire
INFO    pipeline : fenêtre : tout l'historique
INFO    pipeline : livraisons : toutes
INFO    pipeline : étape argent (93f454c5) : début
INFO    pipeline :   argent.usine_horaire           240 240 lignes
INFO    pipeline :   argent.production_horaire       13 104 lignes
```


© 2026 iAlexMG. Tous droits réservés.

```
INFO    pipeline :   argent.energie_horaire          37 223 lignes
INFO    pipeline : étape argent (93f454c5) : fin, 290 567 lignes en 12.0 s
INFO    pipeline : pipeline : terminé
```

Les trois tables du Cours 08, aux mêmes comptes : 240 240, 13 104 et 37 223 lignes. Rien n'a changé dans l'entrepôt, sinon que les séries se refont maintenant par la porte d'entrée, et qu'une ligne de plus s'est inscrite dans `pipeline.executions`.

Étape 2 — Le contrat de l'argent en 1.1.0, et le fuseau du conteneur

Théorie : `Theorie_12_DE.md`, section 4.

Le contrat que Data Engineering tient envers ses consommateurs, `argent_usine`, a un trou : rien n'y dit qu'un jour doit être là. Un mois entier pourrait manquer aux séries de l'usine sans qu'une règle rompe. Ouvrez `contrats\argent_usine.yaml`, passez la version à `1.1.0`, puisque la règle ajoute une promesse sans rien casser :

```yaml
version: 1.1.0
```

Puis ajoutez la règle de fraîcheur en tête de la liste, juste sous `regles:` :

```yaml
  - nom: heures_du_jour_livrees
    dimension: fraicheur
    severite: bloquant
    metrique: periodes_manquantes
    table: argent.usine_horaire
    pas: jour
    doit_etre: 0
```

La métrique du Cours 11 : les jours de la fenêtre qui n'ont aucune heure dans la table. Jugez le contrat sur avril 2026, un mois qui n'existe pas, depuis `01_de` :

```
python -m pipeline contrats --livraisons argent --depuis 2026-04 --jusqua 2026-04
echo %ERRORLEVEL%
```

```
INFO    pipeline : fenêtre : du 2026-04-01 au 2026-04-30
INFO    pipeline : livraisons : argent
INFO    pipeline : étape contrats (00bd6f65) : début
ERROR   pipeline :   ROMPUE     argent_usine.heures_du_jour_livrees                      30   (attendu = 0)
INFO    pipeline :   respectée  argent_usine.une_ligne_par_etiquette_et_par_heure         0   (attendu = 0)
INFO    pipeline :   respectée  argent_usine.couverture_bornee                           0   (attendu = 0)
INFO    pipeline :   respectée  argent_usine.valeur_des_qu_une_seconde_est_couverte      0   (attendu = 0)
INFO    pipeline :   respectée  argent_usine.production_egale_au_cumul                   0   (attendu = 0)
INFO    pipeline :   respectée  argent_usine.moyenne_dans_la_plage_du_capteur            0   (attendu = 0)
ERROR   pipeline : étape contrats (00bd6f65) : échec, 1 règle bloquante rompue : argent_usine.heures_du_jour_livrees
```


© 2026 iAlexMG. Tous droits réservés.

```
ERROR   pipeline : pipeline arrêté : 1 règle bloquante rompue : argent_usine.heures_du_jour_livrees
1
```

Trente jours sans une heure, une règle bloquante, code 1 : la règle voit le mois absent. Jugez maintenant le même contrat là où le DAG le jugera, dans le conteneur d'Airflow. Dans Ubuntu, depuis `plateforme`, pour avril, puis pour le 31 mars seul, comme le ferait un DAG quotidien :

```
docker compose exec airflow-scheduler bash -c "cd /projet/01_de && python -m pipeline contrats --livraisons argent --depuis 2026-04 --jusqua 2026-04"
docker compose exec airflow-scheduler bash -c "cd /projet/01_de && python -m pipeline contrats --livraisons argent --depuis 2026-03-31 --jusqua 2026-03-31"
```

```
INFO    pipeline : fenêtre : du 2026-04-01 au 2026-04-30
…
ERROR   pipeline :   ROMPUE     argent_usine.heures_du_jour_livrees                      29   (attendu = 0)
…
ERROR   pipeline :   ROMPUE     argent_usine.production_egale_au_cumul                    2   (attendu = 0)
…
ERROR   pipeline : pipeline arrêté : 2 règles bloquantes rompues : argent_usine.heures_du_jour_livrees,
argent_usine.production_egale_au_cumul
INFO    pipeline : fenêtre : du 2026-03-31 au 2026-03-31
…
ERROR   pipeline :   ROMPUE     argent_usine.production_egale_au_cumul                    2   (attendu = 0)
…
ERROR   pipeline : pipeline arrêté : 1 règle bloquante rompue : argent_usine.production_egale_au_cumul
```

Le même contrat, sur les mêmes données, rend d'autres verdicts : 29 jours manquants au lieu de 30, et une règle de plus rompue. Et le 31 mars, que Windows jugeait respecté, rompt dans le conteneur, alors que rien n'est faux dans les séries. La cause se voit dans la ligne de commande DuckDB, depuis `Projet_Boreal_intermediaire`, l'entrepôt en lecture seule :

```
ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);
USE entrepot;

SELECT current_setting('TimeZone') AS fuseau, COUNT(DISTINCT heure) AS heures_d_avril,
       MIN(heure) AS premiere
FROM argent.usine_horaire WHERE heure >= DATE '2026-04-01';

SET TimeZone = 'UTC';
SELECT current_setting('TimeZone') AS fuseau, COUNT(DISTINCT heure) AS heures_d_avril,
       MIN(heure) AS premiere
FROM argent.usine_horaire WHERE heure >= DATE '2026-04-01';
```

```
┌──────────────────┬─────────────────┬─────────────────────┐
│      fuseau      │ heures_d_avril  │       premiere      │
```


© 2026 iAlexMG. Tous droits réservés.

varchar	int64	timestamp with time zone
America/Toronto	0	NULL

fuseau varchar	heures_d_avril int64	premiere timestamp with time zone
UTC	4	2026-04-01 00:00:00+00

`heure` est un instant absolu, un `TIMESTAMPTZ` (Cours 08). Comparé à une date, il se lit dans le fuseau de la session : celui de Boréal sous Windows, UTC dans le conteneur. Les quatre dernières heures du 31 mars, de 20 h à minuit à l'heure de l'Est, sont déjà le 1er avril en UTC. Elles tombent dans la fenêtre d'avril, qui compte alors un jour présent, et sortent de celle du 31 mars, dont la production ne retombe plus sur le cumul du compteur. Quittez DuckDB par `.exit`.

La correction pose la fenêtre sur le jour de Boréal, quel que soit le fuseau de la session. Dans `argent_usine.yaml`, remplacez le bloc `fenetre:` par :

```
# Le jour de Boréal, quel que soit le fuseau de la session qui juge.
fenetre:
  colonne: heure AT TIME ZONE 'America/Toronto'
  forme: date
```

`AT TIME ZONE` rend l'heure locale de Boréal, sans fuseau, et c'est elle que le module compare aux bornes. La règle de production faisait déjà ce calcul pour grouper ses journées ; la fenêtre ne le faisait pas. Rejugez dans Ubuntu, avec les deux mêmes commandes :

```
INFO    pipeline : fenêtre : du 2026-04-01 au 2026-04-30
…
ERROR   pipeline :   ROMPUE     argent_usine.heures_du_jour_livrees                        30   (attendu = 0)
…
ERROR   pipeline : pipeline arrêté : 1 règle bloquante rompue : argent_usine.heures_du_jour_livrees
INFO    pipeline : fenêtre : du 2026-03-31 au 2026-03-31
…
INFO    pipeline : étape contrats (53dd3696) : fin, 6 lignes en 0.8 s
```

Trente jours et une règle, le 31 mars respecté : les verdicts de Windows. Jugez enfin le contrat sur tout l'historique, dans l'Anaconda Prompt, depuis `01_de` :

```
python -m pipeline contrats --livraisons argent
```

```
INFO    pipeline :   respectée  argent_usine.heures_du_jour_livrees                       0   (attendu = 0)
INFO    pipeline :   respectée  argent_usine.une_ligne_par_etiquette_et_par_heure         0   (attendu = 0)
INFO    pipeline :   respectée  argent_usine.couverture_bornee                            0   (attendu = 0)
INFO    pipeline :   respectée  argent_usine.valeur_des_qu_une_seconde_est_couverte       0   (attendu = 0)
INFO    pipeline :   respectée  argent_usine.production_egale_au_cumul                    0   (attendu = 0)
```


© 2026 iAlexMG. Tous droits réservés.

```
WARNING pipeline :   à voir     argent_usine.moyenne_dans_la_plage_du_capteur              10   (attendu = 0)
INFO    pipeline : étape contrats (eff369a9) : fin, 6 lignes en 1.6 s
```

Aucun jour ne manque entre octobre et mars, et les dix heures du Cours 11 avertissent toujours. La correction ne change rien à ce qu'un consommateur lit : c'est pourquoi la version passe à 1.1.0, et non à 2.0.0.

Étape 3 — Le DAG de la reconstruction

Théorie : `Theorie_12_DE.md`, section 4.

Tout le chemin froid tient maintenant dans des étapes du pipeline. Un DAG les enchaîne, sans horaire : on le déclenchera à l'étape 5. Depuis `01_de` :

```
notepad dags\boreal_reconstruction.py
```

```
# -*- coding: utf-8 -*-
"""boreal_reconstruction.py : tout le chemin froid de Boréal, refait de zéro, à la demande.

Sept tâches : le bronze, la RAW, le corpus, les séries de l'usine et les contrats, dans l'ordre
des données ; l'ERP par ODBC et la météo par l'API, à côté, sans flèche. Aucun horaire : on le
déclenche quand on veut prouver que le dépôt suffit à refaire la plateforme.
"""
from __future__ import annotations

from datetime import timedelta

import pendulum
from airflow.sdk import dag
from airflow.providers.standard.operators.bash import BashOperator

PIPELINE = "cd /projet/01_de && python -m pipeline"
ENTREPOT = "entrepot"
```

L'en-tête du DAG du Cours 06, en plus court : la porte d'entrée du pipeline, lancée depuis le projet monté, et le pool de l'entrepôt. La suite :

```
@dag(
    dag_id="boreal_reconstruction",
    # Aucun horaire : le DAG se déclenche à la main, par l'interface ou par « airflow dags trigger ».
    schedule=None,
    start_date=pendulum.datetime(2026, 1, 1, tz="UTC"),
    catchup=False,
    max_active_runs=1,
    # Un seul essai de plus, et court : une reconstruction qui échoue se lit, elle ne s'attend pas.
    default_args={"retries": 1, "retry_delay": timedelta(minutes=1)},
```


© 2026 iAlexMG. Tous droits réservés.

```
    tags=["boreal", "chemin froid", "reconstruction"],
    doc_md=__doc__,
)
def boreal_reconstruction():
    def etape(nom: str, commande: str = "") -> BashOperator:
        return BashOperator(task_id=nom, bash_command=f"{PIPELINE} {commande or nom}",
                            pool=ENTREPOT)

    bronze = etape("bronze")
    raw = etape("raw")
    corpus = etape("corpus")
    argent = etape("argent")
    # Sur tout l'historique : une reconstruction doit rendre les verdicts que la livraison annonce,
    # avertissements compris, et aucune règle bloquante ne doit rompre.
    contrats = etape("contrats")

    bronze >> raw >> corpus >> argent >> contrats

    # Les preuves du Cours 04, sans flèche : elles ne nourrissent rien de ce qui précède.
    etape("erp")
    etape("meteo")


boreal_reconstruction()
```

`schedule=None` : aucun horaire, le DAG ne part que si on le déclenche. `max_active_runs=1` : deux reconstructions à la fois n'auraient aucun sens. La fonction `etape` fabrique une tâche par étape du pipeline, toutes dans le pool `entrepot`, puisque toutes l'ouvrent. La chaîne des flèches suit les données, et rien d'autre : l'argent lit le bronze, les contrats lisent tout ce qui précède. L'ERP et la météo ne nourrissent ni la `RAW` ni l'argent : ils se posent à côté, sans flèche. Le pool les fera passer un à un malgré tout, mais un lecteur du DAG voit ce qui dépend de quoi.

Aucune tâche ne passe de fenêtre au pipeline : une reconstruction refait tout l'historique, et les contrats jugent tout l'historique avec elle. Dans Ubuntu, depuis `plateforme`, vérifiez que le fichier se lit, puis attendez qu'Airflow le trouve :

```
docker compose exec airflow-scheduler airflow dags list-import-errors
docker compose exec airflow-scheduler airflow dags list
docker compose exec airflow-scheduler airflow tasks list boreal_reconstruction
```

```
No data found
dag_id                | fileloc                                          | owners  | is_paused | bundle_name | bundle_version
======================+==================================================+=========+===========+=============+===============
boreal_mensuel        | /projet/01_de/dags/boreal_mensuel.py             | airflow | False     | dags-folder | None
boreal_mensuel        | /projet/01_de/dags/boreal_mensuel.py             | airflow | False     | dags-folder | None
boreal_quotidien      | /projet/01_de/dags/boreal_quotidien.py           | airflow | False     | dags-folder | None
```


© 2026 iAlexMG. Tous droits réservés.

```
boreal_reconstruction | /projet/01_de/dags/boreal_reconstruction.py | airflow | False     | dags-folder | None
argent
bronze
contrats
corpus
erp
meteo
raw
```

Un fichier de DAG neuf n'est trouvé qu'à la relecture du dossier, toutes les cinq minutes (Cours 06) : si `boreal_reconstruction` manque à la liste, attendez et relancez `airflow dags list`. Le mensuel y paraît deux fois, une ligne par version de son DAG. Les sept tâches sont là, listées par ordre alphabétique.

Étape 4 — Mettre de côté, puis effacer

Théorie : `Theorie_12_DE.md`, sections 1 et 3.

Avant d'effacer, trois gestes, et chacun a sa raison. D'abord, personne d'autre ne doit écrire pendant qu'on refait : le DAG quotidien part chaque nuit à minuit, et rien ne doit charger une journée dans une plateforme à moitié refaite. Dans Ubuntu, depuis `plateforme` :

```
docker compose exec airflow-scheduler airflow dags pause boreal_quotidien
```

```
dag_id           | is_paused
=================+==========
boreal_quotidien | False
```

La commande affiche l'état d'avant la pause, `False` ; `airflow dags list` montre maintenant `True`. Le mensuel n'a pas besoin de pause : sa borne de fin est passée.

Ensuite, arrêtez la base de séries temporelles :

```
docker compose stop series
```

Le service `series` monte `plateforme/lac/argent/usine`, le dossier des séries du Cours 08. Tant qu'il tourne, ce montage retient le chemin, même après que Windows a effacé le lac, et aucun autre conteneur ne peut plus recréer `plateforme/lac` : l'étape 5 tomberait dès sa première tâche. On le rallumera quand le lac sera revenu.

Enfin, mettez l'entrepôt de côté plutôt que de l'effacer : la deuxième preuve de l'étape 6 le comparera ligne par ligne au refait. Dans l'Anaconda Prompt, depuis `Projet_Boreal_intermediaire`, la ligne de commande DuckDB fermée :

```
move plateforme\entrepot plateforme\avant
rmdir /s /q plateforme\lac
dir /b plateforme
```


© 2026 iAlexMG. Tous droits réservés.

```
        1 répertoires déplacés.
.env
airflow
avant
docker-compose.yml
Dockerfile.airflow
grafana
journaux
mosquitto.conf
```

Le dossier `entrepot` s'appelle maintenant `avant`, avec son `boreal.duckdb` : c'est à la fois la mise de côté et l'effacement de l'entrepôt. Le lac, lui, part en entier : le bronze, l'argent, le corpus déposé, la base de l'ERP lue par ODBC, la météo de l'API, et les tables Iceberg et Delta du Cours 10. Ces dernières ne reviendront pas par le DAG : elles se refont par les huit commandes du Cours 10, si vous en avez besoin. Les sources restent, et la plateforme aussi.

Piège — effacer un dossier qu'un conteneur monte. Si `series` tourne quand le lac disparaît, le DAG de l'étape 5 échoue en un peu plus d'une minute. `bronze`, `erp` et `meteo`, les trois tâches qui n'attendent personne, tombent en une seconde, réessaient une minute plus tard et retombent ; les quatre autres restent `upstream_failed`. Le journal de chacune dit `pipeline arrêté : [Errno 17] File exists: '/projet/plateforme/lac'`. Le dossier n'existe plus sous Windows, et `ls` ne le montre plus dans le conteneur, mais le montage de `series` le retient : un `mkdir` le croit présent. La réparation tient en une commande, `docker compose stop series`, puis un nouveau déclenchement du DAG.

Étape 5 — Refaire

Théorie : `Theorie_12_DE.md`, sections 1 et 4.

Déclenchez la reconstruction, dans Ubuntu, depuis `plateforme` :

```
docker compose exec airflow-scheduler airflow dags trigger boreal_reconstruction
```

La commande rend la main aussitôt, avec une ligne à l'état `queued`. Suivez la reconstruction dans l'interface, http://localhost:8080 (`airflow` et `airflow`) : le DAG `boreal_reconstruction`, son exécution, sa vue en graphe, où les tâches passent au vert une à une. Comptez un peu plus de trois minutes, puis :

```
docker compose exec airflow-scheduler airflow dags list-runs boreal_reconstruction
docker compose exec airflow-scheduler airflow tasks states-for-dag-run boreal_reconstruction manual__2026-09-
23T17:27:12.864785+00:00
```

```
dag_id               | run_id                                    | state   | run_after
=====================+===========================================+=========+==================================
boreal_reconstruction | manual__2026-09-23T17:27:12.864785+00:00 | success | 2026-09-23T17:27:12.864785+00:00

dag_id               | logical_date | task_id  | state
=====================+==============+==========+========
boreal_reconstruction |              | raw      | success
```


© 2026 iAlexMG. Tous droits réservés.

```
boreal_reconstruction |             | corpus    | success
boreal_reconstruction |             | argent    | success
boreal_reconstruction |             | contrats  | success
boreal_reconstruction |             | erp       | success
boreal_reconstruction |             | bronze    | success
boreal_reconstruction |             | meteo     | success
```

Le `run_id` est celui de votre propre liste, avec la date et l'heure de votre déclenchement ; les colonnes des heures de début et de fin sont coupées ici. Une exécution sans horaire n'a pas de date logique : sa colonne est vide. Sept tâches au vert : le chemin froid est revenu.

Combien de temps chacune a pris, le pipeline le sait : c'est la table `pipeline.executions` du Cours 05, dans l'entrepôt refait. Dans la ligne de commande DuckDB, depuis `Projet_Boreal_intermediaire` :

```
ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);
USE entrepot;

SELECT etape, etat, lignes, ROUND(duree_s, 1) AS duree_s
FROM pipeline.executions ORDER BY debut;
```

etape varchar	etat varchar	lignes int64	duree_s double
bronze	réussi	754572	23.3
raw	réussi	580907	19.6
corpus	réussi	36	1.4
argent	réussi	290567	105.1
contrats	réussi	23	28.0
erp	réussi	140295	3.9
meteo	réussi	92	1.9

L'entrepôt refait ne connaît que la reconstruction : sept passages, dans l'ordre du DAG. Le bronze récrit 754 572 lignes, l'historien et la télémétrie ; la `RAW` en charge 580 907 ; l'argent 290 567. Les contrats jugent 23 règles. L'ERP lit par ODBC, depuis le conteneur, les 140 295 lignes de la base rechargée, et la météo tire de l'API 92 jours. L'argent prend à lui seul plus de la moitié du temps : 105 secondes dans le conteneur, contre 12 sous Windows à l'étape 1. Le script relit tout le bronze de l'historien à travers le montage de WSL 2, le coût de la traversée du Cours 06. Quittez DuckDB par `.exit`.

Le lac est revenu, son dossier des séries compris. Rallumez la base de séries temporelles, dans Ubuntu :

```
docker compose start series
```

Étape 6 — Trois preuves

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_12_DE.md`, section 5.

Sept tâches au vert disent que chacune a rendu le code 0, et le Cours 11 a montré ce que vaut ce vert. Trois preuves, de la plus légère à la plus exigeante.

La première est celle d'Analytics Engineering, qui recevra votre `RAW` : le kit de départ, avec son option `--verifier`, compte chaque table contre `LIVRAISON.md` et passe les contrôles de Data Engineering, sans rien écrire. Dans l'Anaconda Prompt, depuis `Data_Domain` :

```
python _univers/kits/demarrer.py --formation 02 --projet Projet_Boreal_intermediaire --verifier
```

```
Kit 02, vérification : le projet C:\…\Data_Domain\Projet_Boreal_intermediaire, comparé à ce que Data Engineering livre
Python 3.11.15, DuckDB 1.5.5 ; rien n'est écrit.

--- 1. La génération des sources
Les sources sont celles de la génération jusqu'à 2026-03.

--- 2. L'état obtenu

Schéma raw : 22 tables, 580 907 lignes
  raw.corpus_index                          34  conforme
  …
  raw.usine_etiquettes                      58  conforme

Schéma argent : 3 tables, 290 567 lignes
  argent.energie_horaire                37 223  conforme
  argent.production_horaire             13 104  conforme
  argent.usine_horaire                 240 240  conforme

Les contrôles de Data Engineering (LIVRAISON.md, sections 2 et 3 ; argent_usine.sql) :
  conforme  chaque table de raw porte ses trois colonnes de lignage
  …
  conforme  l'énergie compte quatre relevés par heure, sauf la dernière

Conforme : chaque table égale ce que Data Engineering annonce, et chaque contrôle passe
(01_Data_Engineering/02_Boreal_intermediaire/Solution/LIVRAISON.md).
Durée totale : 2 s.
```

Vingt-deux tables dans `raw`, trois dans `argent`, douze contrôles : conforme, en deux secondes. La preuve compte, et un compte juste ne dit rien de deux lignes échangées contre deux autres.

La deuxième compare les lignes elles-mêmes, dans les deux sens, contre l'entrepôt mis de côté. Depuis `01_de` :

```
notepad comparer_reconstruction.py
```


© 2026 iAlexMG. Tous droits réservés.

```python
# -*- coding: utf-8 -*-
"""comparer_reconstruction.py : l'entrepôt refait de zéro contre celui d'avant.

Lancement, depuis Projet_Boreal_intermediaire, l'entrepôt libre : python 01_de/comparer_reconstruction.py
"""
from pathlib import Path

import duckdb

REFAIT = Path("plateforme/entrepot/boreal.duckdb")
AVANT = Path("plateforme/avant/boreal.duckdb")
# L'heure du chargement, seule colonne qui diffère par construction, selon le schéma.
HEURE = {"raw": "_charge_le", "pipeline": "charge_le", "argent": None}
# Le journal dans son état : la dernière ligne de chaque fichier.
ETAT = " QUALIFY ROW_NUMBER() OVER (PARTITION BY fichier ORDER BY charge_le DESC) = 1"

con = duckdb.connect()
con.execute(f"ATTACH '{REFAIT.as_posix()}' AS refait (READ_ONLY)")
con.execute(f"ATTACH '{AVANT.as_posix()}' AS avant (READ_ONLY)")

# Les tables livrées, lues dans l'entrepôt d'avant : une table qui manquerait au refait doit se voir.
tables = con.execute("""
    SELECT schema_name, table_name FROM duckdb_tables()
    WHERE database_name = 'avant'
      AND (schema_name IN ('raw', 'argent')
           OR (schema_name = 'pipeline' AND table_name = 'chargements'))
    ORDER BY schema_name, table_name""").fetchall()
```

Les deux entrepôts s'ouvrent en lecture seule, sous deux noms. La liste des tables se lit dans celui d'avant : une table qui manquerait au refait doit se voir, et non disparaître de la comparaison. `HEURE` nomme, par schéma, la seule colonne qui diffère par construction, l'heure du chargement. `ETAT` dit comment lire le journal des chargements : sa dernière ligne par fichier. La suite :

```python
print(f"{len(tables)} tables comparées, sans l'heure de leur chargement\n")
print(f"{'table':<36}{'avant':>10}{'refait':>10}{'écarts':>9}")
ecarts = 0
for schema, table in tables:
    sauf = f" EXCLUDE ({HEURE[schema]})" if HEURE[schema] else ""
    etat = ETAT if schema == "pipeline" else ""
    a = f"(SELECT *{sauf} FROM avant.{schema}.{table}{etat})"
    b = f"(SELECT *{sauf} FROM refait.{schema}.{table}{etat})"
    try:
        n_b = con.execute(f"SELECT COUNT(*) FROM {b}").fetchone()[0]
```


© 2026 iAlexMG. Tous droits réservés.

```
        except duckdb.CatalogException:
            print(f"{schema}.{table:<{35 - len(schema)}}   absente du refait")
            ecarts += 1
            continue
        n_a = con.execute(f"SELECT COUNT(*) FROM {a}").fetchone()[0]
        manque = con.execute(f"SELECT COUNT(*) FROM (FROM {a} EXCEPT ALL FROM {b})").fetchone()[0]
        surplus = con.execute(f"SELECT COUNT(*) FROM (FROM {b} EXCEPT ALL FROM {a})").fetchone()[0]
        ecart_table = manque + surplus
        ecarts += ecart_table
        print(f"{schema + '.' + table:<36}{n_a:>10}{n_b:>10}{ecart_table:>9}")

    # Ce que la reconstruction ne refait pas : les versions remplacées que le journal d'avant raconte.
    remplacees = con.execute("""
        SELECT fichier, COUNT(*) - 1 FROM avant.pipeline.chargements
        GROUP BY fichier HAVING COUNT(*) > 1 ORDER BY fichier""").fetchall()
    print(f"\nHistorique d'avant, que les sources ne portent plus : "
          f"{sum(n for _, n in remplacees)} ligne(s) du journal")
    for fichier, n in remplacees:
        print(f"  {fichier} : {n} version(s) remplacée(s)")

    con.close()
    print(f"\nVerdict : {'0 écart, le dépôt refait la plateforme' if ecarts == 0 else f'{ecarts} écarts'}")
```

`EXCEPT ALL` dans les deux sens, comme au Cours 06 : ce qui manque au refait, et ce qu'il porte en trop, exemplaire par exemplaire. La fin compte à part les lignes du journal d'avant qui racontent une version remplacée d'un fichier. Lancez-le depuis `Projet_Boreal_intermediaire`, la ligne de commande DuckDB fermée :

```
python 01_de/comparer_reconstruction.py
```

```
26 tables comparées, sans l'heure de leur chargement

table                                  avant    refait   écarts
argent.energie_horaire                 37223     37223        0
argent.production_horaire              13104     13104        0
argent.usine_horaire                  240240    240240        0
pipeline.chargements                    1704      1704        0
raw.corpus_index                          34        34        0
…
raw.telemetrie_sorties                204485    204485        0
raw.usine_actifs                          23        23        0
raw.usine_bons_travail                   825       825        0
raw.usine_etiquettes                      58        58        0
```


© 2026 iAlexMG. Tous droits réservés.

```
Historique d'avant, que les sources ne portent plus : 2 ligne(s) du journal
  sources/corpus/index.csv : 1 version(s) remplacée(s)
  sources/usine/gmao/bons_travail.csv : 1 version(s) remplacée(s)

Verdict : 0 écart, le dépôt refait la plateforme
```

Vingt-six tables, zéro écart, en quatre secondes. Le journal d'avant compte pourtant 1 706 lignes, et le refait 1 704 : les deux de trop sont les premières versions de l'export de la GMAO, 779 bons, et du catalogue du corpus, 33 documents, chargées au Cours 03 et remplacées au Cours 06 par la génération jusqu'à mars 2026. Les sources ne les portent plus, et aucune reconstruction ne peut recharger un fichier qui n'existe plus. Comparé ligne pour ligne sans `ETAT`, le journal rendrait 2 écarts, et la preuve échouerait pour une raison qui n'en est pas une.

La troisième relit les verdicts des contrats que la tâche `contrats` a inscrits. Dans la ligne de commande DuckDB, depuis `Projet_Boreal_intermediaire`, l'entrepôt en lecture seule comme à l'étape 5 :

```
SELECT contrat, regle, severite, valeur
FROM pipeline.controles WHERE NOT respectee ORDER BY contrat, regle;
```

contrat varchar	regle varchar	severite varchar	valeur double
argent_usine	moyenne_dans_la_plage_du_capteur	avertissement	10.0
erp_extraits	commande_relivree_a_l_identique	avertissement	609.0
erp_extraits	concessionnaire_connu	avertissement	1.0
usine_historien	capteur_fige	avertissement	1.0
usine_historien	coupures_de_communication	suivi	529.0
usine_historien	points_douteux	suivi	64.0
usine_historien	pression_instable	avertissement	7.0

Seize règles respectées sur vingt-trois, sept hors de leur attendu, et aucune bloquante : les verdicts du Cours 11, à la règle près, et la règle neuve de l'argent à 0. Quittez DuckDB par `.exit`. Reste la preuve du Cours 04, que la tâche `erp` a rendue possible en relisant la base :

```
duckdb -f 01_de/sql/controle_erp_base.sql
```

table_sap varchar	lignes_raw int64	lignes_base int64	raw_sans_base int64	base_sans_raw int64
T005U	20	20	0	0
KNA1	156	156	0	0
MARA	68	68	0	0
MAKT	136	136	0	0

© 2026 iAlexMG. Tous droits réservés.

```
| VBAK    |    27185 |    27185 |           0 |           0 |
| VBAP    |    82842 |    82842 |           0 |           0 |
| CDHDR   |       53 |       53 |           0 |           0 |
| CDPOS   |       53 |       53 |           0 |           0 |
| ZSERIE  |    29782 |    29782 |           0 |           0 |
```

L'état tiré des extraits de la `RAW` refaite égale la base lue par ODBC depuis le conteneur, neuf tables, zéro écart dans les deux sens, à la date de mars 2026.

Point d'arrêt. La première séance est finie : la plateforme se refait par un seul DAG, et trois preuves disent qu'elle est revenue identique. Si vous vous arrêtez ici, laissez le DAG quotidien suspendu ; la seconde séance commence par le chemin chaud.

Étape 7 — Le chemin chaud, de zéro

Théorie : `Theorie_12_DE.md`, section 6.

Le courtier et Kafka tournent depuis le début du labo, la base de séries temporelles depuis la fin de l'étape 5. Dans Ubuntu, depuis `plateforme`, vérifiez-le, puis videz les trois tables du temps réel :

```
docker compose ps mqtt kafka series
docker compose exec series psql -U postgres -d usine -c "TRUNCATE temps_reel.points, temps_reel.alertes, temps_reel.sorties;"
```

Les trois services disent `Up` dans la colonne `STATUS`. `TRUNCATE TABLE` : les tables sont vides, et le reste est en place. Les sujets de Kafka gardent leurs messages du Cours 09, et les deux groupes de consommateurs ont retenu leur décalage au bout de chaque sujet : ils ne liront que ce que le rejeu va publier.

Quatre programmes tournent ensemble, comme à l'étape 13 du Labo 09. Ouvrez quatre Anaconda Prompt. Dans les trois premiers, depuis `01_de`, lancez chacun un programme, dans cet ordre :

```
python -m flux.pont
python -m flux.vers_series
python -m flux.alerte
```

Dans le quatrième, depuis `Data_Domain`, et sans attendre : un programme qui reçoit un message et n'en voit plus d'autre pendant quinze secondes s'arrête.

```
python _univers/rejoueurs/rejouer.py usine --projet Projet_Boreal_intermediaire --depuis 2025-10-13 --jusqua 2025-10-31
```

Le rejeu dure sept minutes et demie. Le terminal de l'alerte reste muet huit jours de Boréal, puis écrit ses lignes au moment où Boréal les aurait vues :

```
Groupe alerte-vibration, alerte à 4.5 mm/s, arrêt à 7.1 mm/s.
2025-10-22 08:15  UTL-CMP2  ALERTE levée : 4.545 mm/s sur la fenêtre de 08:00
2025-10-27 06:15  UTL-CMP2  ARRET levée : 7.554 mm/s sur la fenêtre de 06:00
2025-10-30 14:00  UTL-CMP2  ALERTE retombée : 2.503 mm/s sur la fenêtre de 13:45
2025-10-30 14:00  UTL-CMP2  ARRET retombée : 2.503 mm/s sur la fenêtre de 13:45
```


© 2026 iAlexMG. Tous droits réservés.

Les quatre lignes du Labo 09, à la minute et au millième près. Quinze secondes après la fin du rejeu, chaque programme fait son bilan. Le rejoueur : `62726 messages publiés en 456 s.` Le pont : `usine : relayés vers usine.points : 62726`. Le consommateur :

```
    usine.points : 62726 lus, 62726 insérés, 0 déjà présents
    usine.alertes : 4 lus, 4 insérés, 0 déjà présents
```

Et l'alerte : `points écartés parce qu'arrivés en retard : 0`. Au démarrage du pont ou de l'alerte, une ligne peut commencer par `%4|` et finir par `Coordinator load in progress: retrying` : c'est la bibliothèque de Kafka qui attend que le courtier ait fini de s'éveiller, et elle réessaie seule.

Comptez maintenant ce que la base a reçu. Dans Ubuntu, depuis `plateforme` :

```
docker compose exec series psql -U postgres -d usine
```

```
SET TIME ZONE 'America/Toronto';
-- Attendu : 62 726 points, 58 étiquettes, du 13 octobre à minuit au 31 octobre à 23 h 58.
SELECT count(*) AS points, count(DISTINCT etiquette) AS etiquettes, min(instant), max(instant)
FROM temps_reel.points;
SELECT niveau, etat, fenetre_fin, round(moyenne::numeric, 3) AS moyenne
FROM temps_reel.alertes ORDER BY fenetre_fin, niveau;
```

```
 points | etiquettes |          min           |          max
--------+------------+------------------------+------------------------
  62726 |         58 | 2025-10-13 00:00:00-04 | 2025-10-31 23:58:00-04
(1 row)

 niveau |   etat   |      fenetre_fin       | moyenne
--------+----------+------------------------+---------
 ALERTE | levée    | 2025-10-22 08:15:00-04 |   4.545
 ARRET  | levée    | 2025-10-27 06:15:00-04 |   7.554
 ALERTE | retombée | 2025-10-30 14:00:00-04 |   2.503
 ARRET  | retombée | 2025-10-30 14:00:00-04 |   2.503
(4 rows)
```

Rien de perdu, rien de doublé : 62 726 points, et les quatre changements d'état de l'alerte. Le rejeu a tourné un autre jour que celui du Cours 09, sur une plateforme en partie refaite, et l'alerte tombe à la même minute, parce qu'elle juge l'instant que porte chaque point, et non l'heure à laquelle il arrive. Seule la colonne `recu_le` a changé : elle dit quand le rejeu est passé. Quittez `psql` par `\q`.

Étape 8 — Le tableau de bord de l'usine

Théorie : `Theorie_12_DE.md`, section 7.

© 2026 iAlexMG. Tous droits réservés.

Grafana tourne depuis le début du labo, branché sur la base de séries temporelles par le fichier `plateforme/grafana/sources.yaml`, et il cherche ses tableaux de bord dans `01_de/grafana`, selon `tableaux.yaml`. Ce dossier n'existe pas encore : le tableau de bord est un fichier de votre dépôt, et c'est vous qui l'écrivez. Depuis `01_de` :

```
mkdir grafana
notepad grafana\usine.json
```

Un tableau de bord de Grafana est un document JSON. Son en-tête d'abord :

```
{
  "uid": "boreal-usine",
  "title": "Usine de Val-des-Aulnes : réseau d'air et compresseur 2",
  "description": "Le chemin chaud du Cours 09 de Data Engineering, lu dans le schéma temps_reel de la base de séries temporelles.
Tableau de bord livré au Cours 12 ; Business Intelligence en conçoit la lecture.",
  "tags": ["boreal", "chemin chaud"],
  "timezone": "America/Toronto",
  "editable": false,
  "refresh": "5s",
  "schemaVersion": 39,
  "version": 1,
  "time": {"from": "2025-10-13T00:00:00-04:00", "to": "2025-11-01T00:00:00-04:00"},
  "annotations": {
    "list": [
      {
        "name": "Alertes de vibration",
        "enable": true,
        "iconColor": "red",
        "datasource": {"type": "grafana-postgresql-datasource", "uid": "series"},
        "target": {
          "refId": "Annotations",
          "format": "table",
          "rawQuery": true,
          "editorMode": "code",
          "rawSql": "SELECT fenetre_fin AS time, actif || ' : ' || niveau || ', ' || etat || ' (' || ROUND(moyenne::numeric, 2)
|| ' mm/s)' AS text FROM temps_reel.alertes WHERE $__timeFilter(fenetre_fin) ORDER BY 1"
        }
      }
    ]
  },
```

`uid` est l'identifiant stable du tableau de bord : son adresse, `/d/boreal-usine`, n'en dépend que de lui. `time` fixe la période sur le rejeu d'octobre, et `timezone` le fuseau de Boréal : sans eux, Grafana montrerait les six dernières heures de votre horloge, vides. `editable` à `false` : le tableau de bord s'ouvre en lecture, et il

© 2026 iAlexMG. Tous droits réservés.

se modifie dans ce fichier. `annotations` pose une marque rouge sur chaque courbe, à l'instant de chaque changement d'état de l'alerte, lu dans `temps_reel.alertes`. `$__timeFilter(fenetre_fin)` est une macro de Grafana : elle devient la condition de la période affichée. `refresh` relit la base toutes les cinq secondes : pendant un rejeu, les courbes avancent d'elles-mêmes. Le premier panneau :

```
    "panels": [
      {
        "id": 1,
        "type": "timeseries",
        "title": "Pression au collecteur du réseau d'air (UTL.AIR.PRESSION)",
        "gridPos": {"x": 0, "y": 0, "w": 24, "h": 8},
        "datasource": {"type": "grafana-postgresql-datasource", "uid": "series"},
        "targets": [
          {
            "refId": "A",
            "format": "time_series",
            "rawQuery": true,
            "editorMode": "code",
            "rawSql": "SELECT instant AS time, valeur AS \"pression (kPa)\" FROM temps_reel.points WHERE etiquette =
  'UTL.AIR.PRESSION' AND qualite = 192 AND $__timeFilter(instant) ORDER BY 1"
          }
        ],
        "fieldConfig": {
          "defaults": {
            "unit": "kPa",
            "min": 0,
            "custom": {"lineInterpolation": "stepAfter", "showPoints": "never", "thresholdsStyle": {"mode": "line"}},
            "thresholds": {"mode": "absolute", "steps": [{"color": "green", "value": null}, {"color": "red", "value": 500}]}
          },
          "overrides": []
        },
        "options": {"legend": {"displayMode": "list", "placement": "bottom"}, "tooltip": {"mode": "single"}}
      },
```

Une courbe, la pression au collecteur, en points de bonne qualité seulement, 192. La requête est du SQL de PostgreSQL, écrite en entier dans `rawSql`, où `\"` protège les guillemets du nom de la série. `stepAfter` dessine un palier d'un point au suivant : c'est ainsi que l'historien écrit, par bande morte, une valeur qui tient jusqu'à la suivante (Cours 08). Une ligne rouge marque 500 kPa, le seuil sous lequel le compresseur 3 démarre. Le deuxième panneau, la vibration du compresseur 2 :

```
      {
        "id": 2,
        "type": "timeseries",
        "title": "Vibration du compresseur 2 (UTL.CMP2.VIBRATION), seuils de la procédure : 4,5 et 7,1 mm/s",
```


© 2026 iAlexMG. Tous droits réservés.

```
      "gridPos": {"x": 0, "y": 8, "w": 12, "h": 9},
      "datasource": {"type": "grafana-postgresql-datasource", "uid": "series"},
      "targets": [
        {
          "refId": "A",
          "format": "time_series",
          "rawQuery": true,
          "editorMode": "code",
          "rawSql": "SELECT instant AS time, valeur AS \"vibration (mm/s)\" FROM temps_reel.points WHERE etiquette =
'UTL.CMP2.VIBRATION' AND qualite = 192 AND $__timeFilter(instant) ORDER BY 1"
        }
      ],
      "fieldConfig": {
        "defaults": {
          "unit": "suffix: mm/s",
          "min": 0,
          "custom": {"lineInterpolation": "stepAfter", "showPoints": "never", "thresholdsStyle": {"mode": "line"}},
          "thresholds": {"mode": "absolute", "steps": [{"color": "green", "value": null}, {"color": "orange", "value": 4.5},
{"color": "red", "value": 7.1}]}
        },
        "overrides": []
      },
      "options": {"legend": {"displayMode": "list", "placement": "bottom"}, "tooltip": {"mode": "single"}}
    },
```

Les deux seuils de la procédure d'entretien, 4,5 et 7,1 mm/s, ceux de l'alerte du Cours 09, en lignes orange et rouge. Les deux derniers panneaux, la température de refoulement et le tableau des alertes :

```
    {
      "id": 3,
      "type": "timeseries",
      "title": "Température de refoulement du compresseur 2 (UTL.CMP2.TEMP_REFOULEMENT)",
      "gridPos": {"x": 12, "y": 8, "w": 12, "h": 9},
      "datasource": {"type": "grafana-postgresql-datasource", "uid": "series"},
      "targets": [
        {
          "refId": "A",
          "format": "time_series",
          "rawQuery": true,
          "editorMode": "code",
          "rawSql": "SELECT instant AS time, valeur AS \"température (°C)\" FROM temps_reel.points WHERE etiquette =
'UTL.CMP2.TEMP_REFOULEMENT' AND qualite = 192 AND $__timeFilter(instant) ORDER BY 1"
        }
```


© 2026 iAlexMG. Tous droits réservés.

```
      ],
      "fieldConfig": {
        "defaults": {
          "unit": "celsius",
          "custom": {"lineInterpolation": "stepAfter", "showPoints": "never"}
        },
        "overrides": []
      },
      "options": {"legend": {"displayMode": "list", "placement": "bottom"}, "tooltip": {"mode": "single"}}
    },
    {
      "id": 4,
      "type": "table",
      "title": "Alertes de vibration des compresseurs (temps_reel.alertes)",
      "gridPos": {"x": 0, "y": 17, "w": 24, "h": 7},
      "datasource": {"type": "grafana-postgresql-datasource", "uid": "series"},
      "targets": [
        {
          "refId": "A",
          "format": "table",
          "rawQuery": true,
          "editorMode": "code",
          "rawSql": "SELECT fenetre_debut AS \"fenêtre\", actif, niveau, etat AS \"état\", ROUND(moyenne::numeric, 2) AS
\"moyenne (mm/s)\", recu_le AS \"reçue le\" FROM temps_reel.alertes ORDER BY fenetre_debut, niveau"
        }
      ],
      "fieldConfig": {"defaults": {}, "overrides": []},
      "options": {"showHeader": true}
    }
  ]
}
```

Enregistrez. Grafana relit le dossier toutes les 30 secondes. Ouvrez http://localhost:3000, entrez `admin` et le mot de passe que vous avez mis dans le `.env` de la plateforme, puis le menu des tableaux de bord : le dossier Boréal porte « Usine de Val-des-Aulnes : réseau d'air et compresseur 2 ».

Quatre panneaux, remplis par le rejeu de l'étape 7. La pression monte à 710 kPa environ chaque journée de production, et tombe à zéro la nuit, les fins de semaine et le lundi 13, sans production ; à partir du 20 octobre, la courbe s'épaissit pendant la marche des lignes : c'est l'instabilité que le contrat de l'historien a vue au Cours 11. La vibration du compresseur 2 ne dépasse pas 3 mm/s jusqu'au 17, puis 3,7 le 20, 4,8 le 22, 5,9 le 24, 8,5 le 27 et 9,2 le 28 ; zéro le 29, compresseur arrêté ; 2,5 le 30, réparé. La température de refoulement suit : 79 °C au plus avant la dérive, 99,6 le 27, 103,4 le 28. Trois traits rouges pointillés barrent les courbes, le 22, le 27 et le 30 : les annotations des changements d'état de l'alerte. Le tableau, en bas, porte les quatre lignes de `temps_reel.alertes`.

© 2026 iAlexMG. Tous droits réservés.

Un compte d'administration peut retoucher un panneau à l'écran, le temps d'un essai, mais Grafana refuse de l'enregistrer : `Cannot save provisioned dashboard`. Le tableau de bord vient d'un fichier déclaré par la plateforme, et `allowUiUpdates: false` interdit qu'on l'écrase depuis l'interface. Un changement se fait dans `usine.json`, et Grafana le relit en trente secondes : changez le titre d'un panneau, enregistrez, rafraîchissez la page, puis remettez-le. C'est la règle des DAG, appliquée au tableau de bord : il vit dans le dépôt, et l'outil le lit.

Étape 9 — L'incident, de bout en bout

Théorie : `Theorie_12_DE.md`, section 8.

L'incident de l'automne 2025 se lit maintenant dans ce que la plateforme livre, sans rien lui ajouter. Trois faits, chacun dans une table qu'une autre formation sait lire.

Le premier, la date de l'alerte, vient de la base du temps réel : vous l'avez lu à l'étape 7, le 2025-10-22 à 8 h 15, six jours, deux heures et vingt-neuf minutes avant le déclenchement du compresseur 2, que la GMAO date du 28 octobre à 10 h 44.

Le deuxième, le contrat qui a vu la pression instable, vient de `pipeline.controles`. La reconstruction a jugé tout l'historique d'un coup : elle compte 7 journées, sans dire lesquelles. Rejouez le contrat de l'usine jour par jour sur la fin d'octobre, avec le rejoueur du Cours 11, dans l'Anaconda Prompt, depuis `01_de` :

```
python rejouer_contrats.py usine --depuis 2025-10-13 --jusqua 2025-10-31
```

```
19 tranches jugées, en 13 s.
  points_douteux                  suivi           8 hors cible : 2025-10-13 2025-10-14 2025-10-17 2025-10-18 2025-10-25
2025-10-27 2025-10-28 2025-10-29
  pression_instable               avertissement   7 hors cible : 2025-10-20 2025-10-21 2025-10-22 2025-10-23 2025-10-24
2025-10-27 2025-10-28
  coupures_de_communication       suivi           5 hors cible : 2025-10-14 2025-10-24 2025-10-26 2025-10-27 2025-10-28
  bonne_qualite_dans_la_plage     bloquant        0 hors cible
  capteur_fige                    avertissement   0 hors cible
  etiquette_au_referentiel        bloquant        0 hors cible
  export_du_jour_livre            bloquant        0 hors cible
Alertes envoyées : 7, dont 0 bloquantes.
```

La pression instable avertit dès la journée du 20 octobre, deux jours avant l'alerte de vibration et huit avant le déclenchement, puis chaque jour ouvrable jusqu'au 28. Les points douteux et les coupures, en suivi depuis le Cours 11, s'inscrivent sans rien envoyer : sept alertes, toutes vraies.

Le troisième, la plage des motoneiges touchées, vient de la `RAW`. Dans la ligne de commande DuckDB, depuis `Projet_Boreal_intermediaire`, l'entrepôt en lecture seule :

```
ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);
USE entrepot;
```


© 2026 iAlexMG. Tous droits réservés.

```sql
-- Attendu : 229 motoneiges, assemblées du 2025-10-15 au 2025-10-28 sur ASM-MN.
SELECT COUNT(DISTINCT SERNR) AS motoneiges, MIN(SERNR) AS premiere, MAX(SERNR) AS derniere,
       MIN(ERDAT) AS du, MAX(ERDAT) AS au, STRING_AGG(DISTINCT ARBPL, ', ') AS poste
FROM raw.erp_zserie
WHERE SERNR BETWEEN 'MN26001238' AND 'MN26001466';
```

motoneiges int64	premiere varchar	derniere varchar	du varchar	au varchar	poste varchar
229	MN26001238	MN26001466	20251015	20251028	ASM-MN

Les 229 motoneiges passées au vissage pendant la dérive, sur la ligne d'assemblage des motoneiges, avec leurs dates au format SAP, que la `RAW` garde telles quelles. `STRING_AGG(DISTINCT ARBPL, ', ')` recolle les postes rencontrés en un seul texte, un seul ici (Cours 06 de la formation SQL avancé, section 8). Leurs réclamations, par mois d'ouverture :

```sql
SELECT LEFT(objet->>'date_ouverture', 7) AS mois,
       COUNT(DISTINCT objet->>'reclamation_id') AS reclamations,
       COUNT(*) AS etats,
       COUNT(DISTINCT objet->>'reclamation_id')
           FILTER (WHERE objet->>'cause' = 'REFROIDISSEMENT') AS refroidissement
FROM raw.garanties_reclamations
WHERE objet->>'sernr' BETWEEN 'MN26001238' AND 'MN26001466'
GROUP BY ALL ORDER BY mois;
```

mois varchar	reclamations int64	etats int64	refroidissement int64
2025-12	3	7	1
2026-01	24	46	15
2026-02	29	54	15
2026-03	13	13	5

Chaque réclamation revient à chaque changement de statut (Cours 03) : 69 réclamations, 120 états. Celles du circuit de refroidissement montent en janvier et en février, puis retombent en mars. Le bulletin qui les explique, enfin, dans le corpus :

```sql
SELECT doc_id, titre, chemin, date_publication
FROM raw.corpus_index WHERE doc_id = 'BS-2026-01';
```


© 2026 iAlexMG. Tous droits réservés.

```
┌────────────┬──────────────────────────────────────────────────────────┬──────────────────────────────────┬──────────────┐
│   doc_id   │                          titre                           │              chemin              │ date_publication │
│  varchar   │                         varchar                          │             varchar              │     varchar      │
├────────────┼──────────────────────────────────────────────────────────┼──────────────────────────────────┼──────────────┤
│ BS-2026-01 │ Colliers du circuit de refroidissement insuffisamment serrés │ bulletins/bulletin_bs_2026_01.html │ 2026-02-16   │
└────────────┴──────────────────────────────────────────────────────────┴──────────────────────────────────┴──────────────┘
```

Le bulletin lui-même est dans le lac, déposé tel quel par la tâche `corpus` de la reconstruction : `plateforme/lac/bronze/corpus/bulletins/bulletin_bs_2026_01.html`.

Trois requêtes, trois tables, aucune jointure : Data Engineering ne relie pas les numéros de série aux réclamations, c'est le `STAGING` d'Analytics Engineering qui le fera. Ce que ces requêtes prouvent, c'est que tout ce qu'il faut pour le faire est là, dans la `RAW` refaite, avec le fichier d'où vient chaque ligne. Quittez DuckDB par `.exit`.

Étape 10 — Le `README.md` du dépôt

Théorie : `Theorie_12_DE.md`, sections 3 et 9.

Votre dépôt a onze cours d'histoire, et aucun fichier ne dit à quelqu'un d'autre par où commencer. Depuis `01_de` :

```
notepad README.md
```

Le début : ce qu'est le dépôt, ce qu'il livre, ce qui n'y est pas, ce qu'il suppose.

```
# 01_de — la plateforme de données de Véhicules Boréal inc.

Le dépôt de Data Engineering de Boréal, une entreprise fictive qui fabrique des motoneiges et des
véhicules hors route à l'usine de Val-des-Aulnes. Il fait arriver les données de six systèmes, les
range, les fait tourner à l'heure et prouve qu'elles sont bonnes, par deux chemins : le chemin
froid, qui charge par lots ce que les systèmes livrent, et le chemin chaud, qui écoute l'usine en
continu. Écrit au fil des douze cours de la formation Data Engineering du parcours Data_Domain.

## Ce qu'il livre

| Produit | Où | Pour qui |
|---|---|---|
| La couche `RAW` : 22 tables, en texte, avec leur lignage | schéma `raw` de `plateforme/entrepot/boreal.duckdb` | Analytics Engineering |
| Les séries horaires de l'usine | schéma `argent`, et `plateforme/lac/argent/usine/` | Analytics Engineering, Data Science |
```


© 2026 iAlexMG. Tous droits réservés.

```
| Le temps réel : points, sorties des véhicules, alertes | schéma `temps_reel` de la base `usine` | Business Intelligence |
| Quatre contrats de données, et leurs verdicts | `contrats/`, et `pipeline.controles` | tous les consommateurs |
| Le tableau de bord de l'usine | `grafana/usine.json` | Business Intelligence |

Sur la génération jusqu'à 2026-03 : 580 907 lignes dans `raw`, 290 567 dans `argent`, 23 règles.

## Ce qui n'est pas ici

- Les données : `sources/`, ce que livrent les systèmes, et tout ce qui en dérive, le lac et
  l'entrepôt. Elles vivent à côté du dépôt, dans `Projet_Boreal_intermediaire/`, et se refont.
- Les secrets : les deux `.env`, celui du dépôt et celui de la plateforme. Chacun se copie depuis
  son `.env.exemple`, et Git les ignore.
- La plateforme : le `docker-compose.yml`, l'image d'Airflow et les réglages de Grafana, que
  l'équipe de la plateforme tient dans `plateforme/`.

## Avant de commencer

- `Projet_Boreal_intermediaire/`, avec `sources/` générées jusqu'à 2026-03 et `plateforme/` en place.
- L'environnement conda `formations` : Python 3.11, DuckDB 1.5.5, PyYAML, psycopg, les clients
  MQTT et Kafka ; la ligne de commande DuckDB ; le pilote psqlODBC.
- Docker Engine dans l'Ubuntu de WSL 2, la plateforme démarrée depuis `plateforme/`
  (`docker compose build airflow-init`, puis `docker compose up -d`) et la base de l'ERP chargée.
```

La suite : refaire, prouver, le chemin chaud.

```
## Refaire la plateforme

Tout le chemin froid se refait par un seul DAG, sans horaire. Dans Ubuntu, depuis `plateforme/` :

```bash
docker compose exec airflow-scheduler airflow dags trigger boreal_reconstruction
```

Sept tâches, en un peu plus de trois minutes : `bronze`, `raw`, `corpus`, `argent` et `contrats`
à la suite, puis `erp` et `meteo`, les preuves du Cours 04. Sans Airflow, les mêmes étapes se
lancent depuis `01_de`, dans l'Anaconda Prompt :

```bash
python -m pipeline tout
python -m pipeline argent
python -m pipeline contrats
```
```

© 2026 iAlexMG. Tous droits réservés.

````
## Prouver

Depuis `Projet_Boreal_intermediaire`, la ligne de commande DuckDB fermée :

```bash
python 01_de/comparer_reconstruction.py
duckdb -f 01_de/sql/controle_erp_base.sql
```

Le premier compare l'entrepôt refait à celui qu'on a mis de côté dans `plateforme/avant/` avant
d'effacer : 26 tables, 0 écart dans les deux sens. Le second compare l'état tiré des extraits de
l'ERP à la base lue par ODBC : 9 tables, 0 écart. Les contrats rendent leurs verdicts dans
`pipeline.controles` : aucune règle bloquante rompue sur l'historique.

## Le chemin chaud

Le courtier, Kafka et la base `usine` démarrés, les trois programmes se lancent chacun dans son
terminal, depuis `01_de` : `python -m flux.pont`, `python -m flux.vers_series`,
`python -m flux.alerte`. Le rejoueur publie ensuite octobre 2025, depuis `Data_Domain` :

```bash
python _univers/rejoueurs/rejouer.py usine --projet Projet_Boreal_intermediaire --depuis 2025-10-13 --jusqua 2025-10-31
```

Attendu : 62 726 points, et l'alerte de vibration du compresseur 2 levée le 2025-10-22 à 8 h 15.
Le tableau de bord se lit sur http://localhost:3000, dossier Boréal.
````

Et la carte du dépôt :

````
## Le dépôt

| Dossier ou fichier | Ce qu'il porte |
|---|---|
| `pipeline/` | le chemin froid : une porte d'entrée, `python -m pipeline <étape>`, et une étape par module |
| `sql/` | les scripts SQL : le bronze, les séries de l'usine, la base de séries temporelles, les contrôles |
| `dags/` | les DAG d'Airflow : quotidien, mensuel, et la reconstruction |
| `contrats/` | les contrats de données, en YAML |
| `flux/` | le chemin chaud : l'abonné, le pont vers Kafka, le consommateur, l'alerte |
| `tables/` | les tables Iceberg et Delta du lac |
| `spark/` | la comparaison de Spark et de DuckDB, qui n'entre dans aucun chemin |
| `grafana/` | le tableau de bord de l'usine |
| `postmortems/` | les incidents de données, écrits sans blâme |
| `tests/` | les tests du pipeline et du chemin chaud : `pytest`, 8 tests, sans données ni réseau |
````


© 2026 iAlexMG. Tous droits réservés.

```
| `config.toml`, `.env.exemple` | la configuration, et le modèle des secrets |
| `comparer_rattrapage.py`, `comparer_reconstruction.py`, `rejouer_contrats.py` | les preuves |
```

Relisez-le comme le lirait quelqu'un qui ne vous connaît pas. Il ne raconte pas le code, que les en-têtes des fichiers expliquent : il dit quoi avoir avant de commencer, quoi taper, dans quel ordre, et ce qui doit sortir. Il dit aussi ce qui n'est pas dans le dépôt, et pourquoi. Le détail de chaque produit, table par table, reste dans `LIVRAISON.md`, que la formation tient à jour depuis le Cours 03, et le détail de chaque promesse dans les contrats.

Étape 11 — Versionner, et étiqueter la version 1.0

Théorie : `Theorie_12_DE.md`, section 9.

Depuis `01_de` :

```
git add pipeline/argent.py pipeline/__main__.py contrats/argent_usine.yaml
git add dags/boreal_reconstruction.py comparer_reconstruction.py
git add grafana/usine.json README.md
git commit -m "Cours 12 : la reconstruction de zero, ses preuves, le tableau de bord et le README"
git tag -a v1.0 -m "La plateforme de Boreal, refaite de zero et prouvee"
git log --oneline --decorate -1
```

```
67ad5e5 (HEAD -> main, tag: v1.0) Cours 12 : la reconstruction de zero, ses preuves, le tableau de bord et le README
```

Votre empreinte de commit sera autre. Sept fichiers entrent : deux modules du pipeline, le contrat, le DAG, la comparaison, le tableau de bord et le `README.md`. Rien de ce que la reconstruction a refait n'y entre : le lac, l'entrepôt et `plateforme/avant/` restent hors du dépôt, comme depuis le Cours 01.

Une étiquette annotée, `-a`, porte un auteur, une date et un message, comme un commit : c'est la forme qu'on donne à une version livrée. `v1.0` nomme maintenant cet état du dépôt, et il le nommera encore quand `main` aura avancé. Si votre dépôt a un serveur distant, l'étiquette ne part pas avec `git push` : elle demande `git push origin v1.0`.

Le DAG quotidien reste suspendu : les sources s'arrêtent au 31 mars 2026, et il échouerait chaque nuit sur ses règles de fraîcheur, avec raison (Cours 11).

Récapitulatif

Ce que vous avez mesuré	**Le chiffre**
La plateforme reçue	l'image d'Airflow de 3,18 à 3,27 Go, `PostgreSQL Unicode` dedans ; la base de l'ERP à mars 2026, `VBAK` 27 185 et `ZSERIE` 29 782
L'étape `argent`	240 240, 13 104 et 37 223 lignes : 290 567 en 12 s sous Windows

© 2026 iAlexMG. Tous droits réservés.

Ce que vous avez mesuré	Le chiffre
Le fuseau du conteneur	avril 2026 : 30 jours et une règle sous Windows, 29 jours et deux règles dans le conteneur ; le 31 mars rompu dans le conteneur seul ; après correction, les mêmes verdicts des deux côtés
La reconstruction	sept tâches au vert en 3 min 12 s, dont 105 s pour l'argent ; 754 572 lignes de bronze, 580 907 de `RAW`, 290 567 d'argent, 23 règles, 140 295 lignes de l'ERP, 92 jours de météo
Le lac monté par `series`	`[Errno 17] File exists: '/projet/plateforme/lac'` tant que le service tourne
Les comptes	le kit conforme : 22 tables dans `raw`, 3 dans `argent`, 12 contrôles, 2 s
Les lignes	26 tables, 0 écart dans les deux sens ; 2 lignes du journal d'avant que les sources ne portent plus
Les contrats	23 règles, 7 hors de leur attendu, aucune bloquante : les verdicts du Cours 11
L'ERP	9 tables, 0 écart dans les deux sens
Le chemin chaud	62 726 points publiés en 456 s, relayés et insérés, 0 déjà présent ; l'alerte levée le 2025-10-22 à 8 h 15, l'arrêt le 27 à 6 h 15, les deux retombés le 30 à 14 h
Le tableau de bord	quatre panneaux ; la vibration de 3 à 9,2 mm/s, la température de 79 à 103,4 °C ; `Cannot save provisioned dashboard` depuis l'interface
L'incident	la pression instable dès le 20 octobre, 7 journées ; 229 motoneiges assemblées du 15 au 28 octobre ; 69 réclamations, dont 36 du circuit de refroidissement ; le bulletin du 2026-02-16
La livraison	le `README.md` du dépôt, sept fichiers commités, l'étiquette `v1.0`

Ce que vous savez maintenant et que vous ne saviez pas : une plateforme n'est finie que le jour où on l'a effacée et refaite ; changer de machine fait tomber ce qu'aucun test ne voyait, jusque dans un contrat ; refaire n'est pas prouver, et une preuve honnête compte à part ce qu'elle ne peut pas refaire ; un flux se rejoue, et l'alerte revient à la même minute parce qu'elle juge le temps de l'événement ; un tableau de bord est un fichier du dépôt ; et la `RAW` porte de quoi remonter un incident sans rien demander à personne.

© 2026 iAlexMG. Tous droits réservés.

La formation s'arrête ici. Votre `RAW` et vos séries sont ce que la formation Analytics Engineering reçoit, par le kit 02 ou par votre propre projet, que son Cours 00 ramène d'abord à la génération par défaut, parce qu'elle reçoit les mois de 2026 à son Cours 07 ; le tableau de bord, ce que Business Intelligence reprend ; les séries de l'usine, ce que Data Science modélise. Et le dépôt `01_de`, étiqueté `v1.0`, est la pièce que vous pouvez montrer.

Dépannage

Symptôme	Cause	Ce qu'on fait
`level=warning msg="The \"GRAFANA_MOT_DE_PASSE\" variable is not set. Defaulting to a blank string."`	le `.env` de la plateforme n'a pas la ligne de Grafana	l'ajouter, puis `docker compose up -d grafana`
Grafana refuse le mot de passe du `.env`	le volume de Grafana existait déjà quand la ligne a été ajoutée : le mot de passe ne s'applique qu'à sa création	`docker compose rm -sf grafana`, `docker volume rm boreal_grafana_donnees`, puis `docker compose up -d grafana` ; la source et le tableau de bord reviennent de leurs fichiers
`pyodbc.drivers()` ne rend que `['ODBC Driver 18 for SQL Server']`, ou `No module named 'pyodbc'`	l'image n'a pas été refaite, ou le `Dockerfile.airflow` n'a pas été recopié	recopier le fichier, `docker compose build airflow-init`, puis `docker compose up -d`
`Étape inconnue : argent.`	la ligne de `ETAPES` ou l'import manquent dans `pipeline\__main__.py`	les trois changements de l'étape 1
`Binder Error: Unique file handle conflict` pendant l'étape `argent`	le script est exécuté sans passer par `adapter`	`con.execute(adapter(...))`, comme dans le labo
`pipeline arrêté : argent_usine.sql n'attache plus l'entrepôt comme au Cours 08`	la ligne de l'`ATTACH` du script du Cours 08 a changé	lui rendre sa forme exacte, `ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot;`
`EntrepotOccupe: L'entrepôt est occupé : fermez la ligne de commande DuckDB, puis relancez.`	la ligne de commande DuckDB tient l'entrepôt ouvert	`.exit` dans DuckDB, puis relancer
Le contrat de l'argent rend 29 jours sur avril dans le conteneur, et `production_egale_au_cumul` rompue	la fenêtre porte encore sur `heure`	`colonne: heure AT TIME ZONE 'America/Toronto'`, étape 2

© 2026 iAlexMG. Tous droits réservés.

Symptôme	Cause	Ce qu'on fait
`boreal_reconstruction` manque à `airflow dags list`	un fichier de DAG neuf n'est trouvé qu'à la relecture du dossier, toutes les cinq minutes	`airflow dags list-import-errors`, qui nomme la ligne fautive ; sinon, attendre
La reconstruction échoue en une minute, `[Errno 17] File exists: '/projet/plateforme/lac'`	`series` tournait quand le lac a été effacé	`docker compose stop series`, puis déclencher de nouveau
La tâche `erp` échoue sur une connexion refusée à `localhost`, port 5433	le compose n'a pas été recopié : le conteneur ne connaît pas `PIPELINE_ERP_SERVEUR`	recopier `docker-compose.yml`, puis `docker compose up -d`
`comparer_reconstruction.py` rend 2 écarts sur `pipeline.chargements`	la ligne `ETAT` manque, ou n'est pas appliquée au journal	la comparaison du journal se fait sur sa dernière ligne par fichier, étape 6
`comparer_reconstruction.py` s'arrête sur `IO Error` pour `plateforme/avant/boreal.duckdb`	l'entrepôt a été supprimé au lieu d'être déplacé, ou le script n'est pas lancé depuis `Projet_Boreal_intermediaire`	lancer depuis `Projet_Boreal_intermediaire` ; un entrepôt supprimé ne se retrouve pas, et il ne reste que les comptes et les contrats
Un programme du chemin chaud s'arrête avant le rejeu sur `Aucun message depuis 15 s : arrêt.`	il a reçu des messages restés d'une séance précédente, puis quinze secondes de silence	le relancer, et lancer le rejeu dans la foulée
`usine.points : … déjà présents` n'est pas à 0	les tables du temps réel n'ont pas été vidées	le `TRUNCATE` de l'étape 7, puis un nouveau rejeu
Le tableau de bord s'ouvre vide	la période a été changée dans l'interface, ou l'étape 7 n'a pas tourné	rouvrir le tableau de bord depuis le dossier Boréal, qui rend la période du fichier ; sinon, refaire l'étape 7
`Cannot save provisioned dashboard`	c'est voulu : le tableau de bord vient d'un fichier	modifier `01_de/grafana/usine.json`

© 2026 iAlexMG. Tous droits réservés.